PAST Framework

Tuning the Persona, Not the Weights: The PAST Framework for Personality-Adaptive Multi-Agent Trading Systems

Author: John Tytko — Creator and Author **Date:** July 2026 **Version:** 2.1

Abstract

Traditional alignment methodologies for Large Language Models (LLMs) rely heavily on parameter fine-tuning (e.g., LoRA, RLHF), which introduces prohibitive computational overhead and compromises the foundational generalization capabilities of the base model. To address this, we introduce the **Personality-Adaptive Score Tuning (PAST)** framework, a novel, weight-free orchestration layer for Multi-Agent Systems. Instead of altering neural network weights, PAST quantifies desired behavioral personas using a multi-dimensional Likert scale (1–7). These numerical personality profiles directly cascade into dynamic, prompt-level execution rules and scoring functions. A supervisory "Coach" agent evaluates operational KPIs on a bi-weekly cycle, iteratively tuning the Likert scores within a trust region to alleviate system bottlenecks and optimize performance. Operational agents autonomously exploit historical runtime logs to refine their behaviors *in-context* without backpropagation. We implement PAST in a four-agent trading system (two competing traders, an independent judge, and a coach) and evaluate it across 6 tickers over 6.5 months using historical price data. The system achieves +15.3% cumulative returns (+$763.27 on $5,000), with agents producing differentiated behavior on identical instruments — exactly as their personality scores predict. Our framework decouples agent behavioral alignment from model training, offering a highly resource-efficient, explainable, and fully reversible architecture for steerable AI systems.

---

1. Introduction

The deployment of Multi-Agent Systems (MAS) powered by Large Language Models (LLMs) has emerged as a dominant paradigm for solving complex, distributed tasks. By coordinating specialized agents to collaborate or compete, these systems simulate human workflows to execute multi-step objectives. However, controlling and aligning behavioral dynamics within a multi-agent collective remains a significant open challenge.

Current methods for altering or aligning agent behavior heavily rely on parameter-level adjustments. Techniques such as Full Fine-Tuning, Low-Rank Adaptation (LoRA), and Reinforcement Learning from Human Feedback (RLHF) modify the underlying neural network weights to achieve stylistic or behavioral shifts. While effective for single-model alignment, parameter fine-tuning is fundamentally ill-suited for the dynamic, iterative requirements of multi-agent orchestration due to three primary limitations:

- **Prohibitive Compute Costs**: Retraining or updating weights across an entire collective of heterogeneous agents demands massive hardware and time commitments.
- **Loss of Model Generalization**: Fine-tuning a model to exhibit a narrow behavioral trait frequently causes catastrophic forgetting, degrading its baseline reasoning capabilities.
- **Irreversibility**: Once neural network weights are updated, reversing a behavioral shift requires reverting to past checkpoints or initiating a fresh training cycle, eliminating real-time agility.

To bypass these limitations, practitioners frequently turn to prompt engineering and hardcoded system prompts to establish agent personas. While compute-efficient, this approach is fundamentally static. It lacks an evolutionary mechanism to self-correct when behavioral conflicts, communication bottlenecks, or operational inefficiencies arise during runtime.

To bridge the gap between expensive weight modification and rigid prompt engineering, we introduce the **Personality-Adaptive Score Tuning (PAST)** framework. PAST is a novel, weight-free alignment layer that formalizes agent behaviors into quantified human personality dimensions via a standard 1–7 Likert scale. Instead of modifying network parameters, these quantified personality vectors cascade directly into runtime constraints, prompt modifications, and evaluation metrics at the model's inference boundary.

The core contribution of this work is the decoupling of behavioral adaptation from neural network optimization. Under the PAST architecture, an autonomous, supervisory "Coach" agent audits system-wide Key Performance Indicators (KPIs) and operational logs on a bi-weekly cadence. Identifying systemic bottlenecks, the Coach shifts the agent's numerical Likert vector within a trust region (±1 per cycle). This adjustment immediately alters downstream behavior without backpropagation. Concurrently, operational units utilize historical runtime logs to guide their in-context adaptation, effectively learning from their own execution "PAST."

This paper makes the following contributions:

1. **The PAST framework** — a formal, mathematical definition of personality scores as Likert vectors, their mapping to operational constraints via a rule cascade, and the coach-mediated tuning loop with trust region. Conceptualized by the author as a weight-free alternative to RLHF/DPO for multi-agent behavioral alignment.
2. **In-Context Log Exploitation (ICLE)** — a micro-level feedback mechanism where agents learn from their own execution history at the inference boundary
3. **A four-agent competitive trading architecture** — two traders with opposite strategies, an independent judge, and a coach — implemented on heterogeneous models (GLM-5.2, Deepseek R1, Claude Fable 5)
4. **Empirical evaluation** — a backtest across 6 tickers and 6.5 months showing +15.3% returns with measurable behavioral diversity, demonstrating that PAST scores produce predictable, differentiated agent behavior on identical instruments

While this paper validates PAST in the financial trading domain, the framework is domain-agnostic. The same personality tuning mechanism — quantified Likert scores, coach-mediated bi-weekly adjustment, in-context log exploitation — can be applied to any multi-agent system requiring behavioral adaptation, from AI tutors (where teaching style is tuned to student learning outcomes) to software engineering agents (where code review strictness is tuned to project KPIs).

---

2. Related Work

2.1 Multi-Agent Orchestration Frameworks

Modern multi-agent architectures rely on frameworks like Microsoft's AutoGen, CrewAI, and LangGraph to coordinate autonomous collectives. AutoGen focuses on conversation-driven workflows via customizable agents. CrewAI structures agents into production-oriented "crews" with defined roles, tasks, and tool access. LangGraph introduces cyclic graph topologies for complex state machines.

While these frameworks excel at infrastructure, they lack native mechanisms for dynamic, self-correcting behavioral alignment. Agent personas are rigidly codified at initialization via static system prompts. If a persona creates an operational bottleneck, resolving the issue requires manual prompt rewriting and system restart. PAST supplements these frameworks by introducing an algorithmic layer that translates quantified metrics into fluid prompt adjustments, allowing agent behaviors to evolve programmatically.

2.2 Multi-Agent LLM Trading Frameworks

TradingAgents (Tauric Research, 80k+ GitHub stars) simulates a trading firm with 7 specialized LLM agents (fundamental, sentiment, news, technical analysts + bull/bear researchers + trader + risk manager). Agents debate before reaching consensus. Published results show 24.9% annualized returns with Sharpe ratios of 1.69-2.11. The framework uses static agent roles — no personality tuning.

AI Hedge Fund (virattt, 51.7k stars) deploys 18 agents embodying legendary investor personas (Buffett, Munger, Burry, Wood). Each agent has a fixed investment philosophy encoded in its prompt. Explicitly a simulation — no real trades execute. No personality adaptation.

AI-Trader (HKUDS, 21k stars) provides agent-native trading with a "team challenge mode" for competitive agents. Closest to our competitive architecture but lacks a coaching/tuning layer.

2.3 Psychometric Profiling and LLM Personas

A growing body of literature explores the capacity of LLMs to simulate human personality structures using frameworks like the Big Five (OCEAN) or Myers-Briggs Type Indicator (MBTI). Researchers have demonstrated that prompting LLMs with specific trait descriptions alters their decision-making risk profiles, linguistic styles, and negotiation behaviors.

However, existing research predominantly treats LLM psychometrics as an evaluative curiosity rather than an operational tuning knob. Current implementations rely on long, descriptive prompt paragraphs to induce a persona, consuming significant context windows and suffering from high prompt sensitivity. PAST formalizes persona engineering by converting raw qualitative text into structured 1–7 Likert vectors, mapping numeric scores directly to inference-boundary constraints.

2.4 Parameter-Efficient vs. Weight-Free Alignment

Traditional alignment modifies model weights to enforce preferences. Direct Preference Optimization (DPO) and RLHF adjust token probabilities at the parameter level. While effective, parameter tuning is computationally prohibitive for runtime multi-agent optimization, lacks real-time reversibility, and triggers catastrophic forgetting.

Parameter-Efficient Fine-Tuning (PEFT) techniques like LoRA isolate weight updates to compact adapter modules. Even with LoRA, switching an agent's persona mid-task requires hot-swapping weight matrices, incurring latency penalties and requiring GPU orchestration. PAST introduces a completely **weight-free alignment** paradigm — the tuning target shifts entirely from neural network parameters (W) to structured prompting scores (P), achieving real-time behavioral plasticity through API boundaries while maintaining a frozen, highly generalized base model.

---

3. The PAST Framework

3.1 The Likert Personality Vector

Let P represent the personality profile of an agent, defined as a multi-dimensional vector in a discrete parameter space:

$$P = [P_1, P_2, \ldots, P_n]^T, \quad \text{where } P_i \in \{1, 2, \ldots, 7\}$$

Each dimension i corresponds to an explicit behavioral attribute. In the Trading Arena implementation, the core dimensions are:

Dimension (i)	Attribute	Trading Rule Impact
P_1	Risk Tolerance	Stop loss percentage ($P_1 = 3 \rightarrow 3\%$ stop, $P_1 = 6 \rightarrow 10\%$ stop)
P_2	Conviction	Position sizing and hold duration
P_3	Impulsivity	Entry speed (act immediately vs. wait for confirmation)
P_4	Patience	Entry threshold strictness (stricter filter = fewer trades)

Additional dimensions can be defined (Adaptability, Technical Focus, Sector Specialization, Position Concentration) but the four core scores are sufficient to produce differentiated behavior.

3.2 The Rule Cascade and Mapping Functions

The core of the weight-free alignment layer is the mapping function $f(P_i)$, which translates a single scalar Likert score into an operational, text-based system constraint (C_i). This mapping uses a linear scaling operation to determine a constraint severity multiplier, α_i:

$$\alpha_i = \frac{P_i - 1}{6}, \quad \text{where } \alpha_i \in [0, 1]$$

This coefficient scales both the quantitative execution parameters and the dynamic prompt templates injected into the context window. The global system prompt (S) presented to the agent at runtime is constructed via a concatenation of the base task role (R) and the active personality constraints:

$$S = R \ \cup \ \bigcup_{i=1}^{n} \Phi(\alpha_i, T_i)$$

Where T_i represents a structured prompt matrix mapping extreme traits ($P_i = 1$ vs. $P_i = 7$) to clear behavioral rules, and Φ is the dynamic text compiler.

Example cascade (Trading Arena): - P_4 (Patience) = 6 $\implies$ $\alpha_4 = 0.83$ $\implies$ strict entry filter: RSI < 35 AND rising 50-day MA - P_4 (Patience) = 7 $\implies$ $\alpha_4 = 1.0$ $\implies$ stricter filter: RSI < 20 AND rising 50-day MA AND bullish candle confirmation

A single integer change (6 → 7) cascades through the rule engine to tighten the entry threshold, reducing trade frequency — exactly the behavior observed in our backtest when Rocky adjusted Alpha's Patience.

3.3 Mathematical Formulation of the Coach Review Loop

Let $\mathbf{P}_t = [P_1, P_2, \ldots, P_n]^T$ be the personality vector at bi-weekly interval t. The optimization objective of the supervisory Coach agent is to minimize the variance between target system KPIs (K_{target}) and observed runtime metrics ($K_{\text{observed},t}$). At the end of each two-week window, the Coach updates each personality trait score using the following clipped transition function:

Extra \left or missing \right

Where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer to maintain strict Likert boundaries. The delta adjustment is:

$$\Delta P_{i,t} = \gamma \cdot \sigma_i \cdot \Phi(K_{\text{target}} - K_{\text{observed},t})$$

Where: - $\gamma \in (0, 1]$ is the Coach's *adaptation velocity* (learning rate). - $\sigma_i \in \{-1, 1\}$ is a directional sign vector derived by the Coach's semantic reasoning layer, dictating whether increasing or decreasing trait i mitigates the observed bottleneck. - $\Phi(\cdot)$ is a normalized scaling function mapping the performance deficit to a scalar shift.

The trust region constraint ($|\Delta P_{i,t}| \leq 1$) is inspired by Proximal Policy Optimization's clipped objective (Schulman et al., 2017). It ensures that personality evolves gradually rather than swinging wildly based on short-term results — a critical stability property for

systems handling real capital.

3.4 PAST vs DPO vs RLHF

Dimension	DPO / RLHF	LoRA / PEFT	PAST
What is tuned	Model weights	Adapter weights	Likert scores → prompts
Requires	GPU training, preference data	GPU training, adapter modules	CPU-only, observation of outcomes
Granularity	Model-level	Model-level	Agent-level (per-agent)
Reversibility	Difficult (retrain)	Moderate (swap adapter)	Trivial (adjust score)
Compute	Hours to days	Minutes to hours	Instant
Auditability	Opaque (weights)	Opaque (adapters)	Full audit trail (logged score changes)
Catastrophic forgetting	Yes (risk)	Reduced	None (base model frozen)

PAST does not replace RLHF/DPO/LoRA — it operates at a different abstraction level. These methods tune the model; PAST tunes the instructions given to the model. They are complementary.

---

4. In-Context Log Exploitation (ICLE)

While the supervisory Coach agent establishes macro-level behavior via bi-weekly Likert vector shifts, operational units execute micro-level self-correction via **In-Context Log Exploitation (ICLE)**.

4.1 The ICLE Feedback Loop

Under traditional architectures, an agent approaches each execution task as an isolated, stateless event. Under PAST, agents are explicitly bound to their execution history. The ICLE mechanism functions as a localized, non-gradient optimization loop:

1. **Chronological Log Compaction:** The agent's runtime wrapper maintains an append-only transaction log containing requests, reasoning paths, tool calls, outputs, and execution scores. When an agent initiates a task, a retrieval mechanism extracts

the K most relevant log traces. These are passed through a summarization layer to strip token noise while preserving explicit fault lines (exceptions, rejections, timeouts).

2. **Self-Contrastive Inference:** The agent ingests the compacted history alongside its freshly tuned Likert rules. It evaluates its historical trajectory via an internal meta-prompt: *"Identify where past behaviors violate the boundary limits of the newly assigned personality metrics."*

3. **Behavioral Shifting:** The agent modifies its immediate execution strategy based on this self-contrastive analysis. For instance, if Alpha reads that its Patience score was increased from 6 to 7 by the Coach, and its execution log reveals that previous trades at RSI 35-40 resulted in losses, it explicitly suppresses entries in the 35-40 range in the current context window.

4.2 Mathematical Formalization of ICLE Context

Let H_t be the historical log trace up to task t, where each entry $h \in H_t$ contains the tuple of prompt, response, and performance feedback: $h = (x, y, r)$. The operational context window C_t passed to the underlying frozen LLM is:

$$C_t = S(\mathbf{P}_t) \cup \Psi(H_t, K) \cup x_t$$

Where $S(\mathbf{P}_t)$ is the system prompt generated by the Likert vector cascade, x_t is the current task input, and $\Psi(H_t, K)$ is a filtering operator that compresses the historical log down to the K most critical behavioral failure vectors. By injecting $\Psi(H_t, K)$, the agent performs in-context learning over its own operational timeline — learning from its PAST at the inference boundary.

4.3 Trading Arena Implementation

In the Trading Arena, ICLE is implemented via file-based memory:

File	ICLE Role	Updated By
`state/agent_alpha/state.md`	Alpha's trade history, P&L, strategy learnings	Alpha (after each trade)
`state/agent_beta/state.md`	Beta's trade history, P&L, strategy learnings	Beta (after each trade)
`state/judge_decision_log.md`	Judge verdicts (append-only audit trail)	Judge
`state/rocky/week_N_notes.md`	Coach's bi-weekly tuning notes	Rocky

At the start of each trading session, agents read their state file (containing the K most recent trades and learnings) via `read_file()`. This is the $\Psi(H_t, K)$ operator — compressed history injected into the context window. The agent then self-evaluates: "My last 3 trades at RSI 35-40 all lost. My Patience is now 7. I will require RSI < 20 before entering." This is ICLE in action.

5. System Architecture

5.1 Four-Agent Design

The Trading Arena deploys four agents with distinct roles, models, and providers:

Agent	Role	Model	Provider	Location
Alpha	Trader (Renaissance)	GLM-5.2	Z.AI API	Cloud
Beta	Trader (Aschenbrenner)	Deepseek R1 70B	Ollama	Local
Judge	Independent verifier	Claude Fable 5	Claude Code	Cloud
Rocky	Coach (PAST tuning)	Claude Fable 5	Claude Code	Cloud

Design principle: cognitive diversity. Alpha and Beta run on different models from different providers. Alpha uses a 744B MoE model (GLM-5.2) optimized for long-context reasoning. Beta uses a 70B dense model (Deepseek R1) optimized for local inference. The Judge and Coach use Anthropic's Claude (Fable 5) for independent verification. This three-model diversity prevents single-model bias — no two agents share the same "brain."

5.2 Agent Personalities

Agent Alpha — Renaissance (Jim Simons style):

$$\mathbf{P}_{\text{Alpha}} = [3, 4, 2, 6]^T \quad (\text{Risk, Conviction, Impulsivity, Patience})$$

Strategy: Statistical mean reversion. Buys oversold stocks with rising 50-day MAs. Holds 3-5 days. 3% stop loss. Sector-agnostic.

Agent Beta — Aschenbrenner style:

$$\mathbf{P}_{\text{Beta}} = [6, 7, 5, 7]^T \quad (\text{Risk, Conviction, Impulsivity, Patience})$$

Strategy: AI infrastructure thesis. Buys second-order beneficiaries (power, nuclear, neocloud). 10% stop loss. Weeks-months holds.

5.3 The Judge (13-Point Checklist)

The Judge is a separate LLM that evaluates every trade proposal before execution. Key checklist items include:

1. Thesis supported by data
2. Position size appropriate
3. Stop loss set
4. PAST drift check (within trust region)

5. Convergence check (both agents in same ticker?)
6. Earnings within 5 days
7. Liquidity check
8. PAST personality fit
9. Options budget check (for Beta)
10. Catalyst present
11. State file honesty 11b. Restricted stock check 11c. **IPO ban** (HARD BLOCK — no stocks public < 90 days) 11d. PAST drift detection

Separation of powers: The Judge runs on a different model (Claude Fable 5) from both traders. It cannot be influenced by either agent's reasoning — it pulls its own data and applies the checklist independently. This is analogous to the separation between prosecution and judiciary in legal systems.

---

6. Experimental Setup

6.1 Backtest Configuration

Parameter	Value
Date range	January 1, 2026 — July 21, 2026 (6.5 months)
Tickers	SPCX, OKLO, NVDA, APLD, CCJ, MU
Allocation	$2,500 per agent ($5,000 total) || Data source | Yahoo Finance (via yfinc − day MA slope (5 − day lag), 52 − week high / low || Judge threshold | ≥ 10/13 required for approval || Rocky tuning cycle | Every 10 trading d

6.2 Ticker Selection

Tickers were selected to span different market regimes and thesis categories:

Ticker	Category	Why Selected
SPCX	IPO (June 2026)	Tests IPO behavior — agents should avoid or lose
OKLO	AI infra (nuclear/SMR)	Core Aschenbrenner thesis — power bottleneck
NVDA	AI infra (chips)	Beta's thesis-banned stock (short side only)
APLD	AI infra (data centers)	Second-order beneficiary
CCJ	AI infra (uranium)	Adjacent to nuclear thesis
MU	Semiconductors (memory)	AI training demand — high volatility

6.3 Agent Rules

Alpha (Mean Reversion): - **Primary entry:** RSI < 35 AND 50-day MA rising AND price > 85% of 50-day MA - **Secondary entry:** RSI 35-45 AND 50-day MA rising AND price > 90% of 50-day MA (Rocky-approved, reduced 8% position size) - **Exit:** 3% stop loss OR RSI > 55 OR 5-day time limit - **Position size:** 15% of allocation (8% for secondary entries)

Beta (Thesis-Driven): - **Entry:** Price within 25% of 52-week low (thesis intact) - **Exit:** 10% stop loss (no time limit — weeks to months) - **Position size:** 20% of allocation

---

7. Results

7.1 Overall Performance

Metric	Alpha ($P = [3, 4, 2, 6]$)	Beta ($P = [6, 7, 5, 7]$)	Combined
Net P&L	+$8.86	+$754.41	+$763.27
Return	+0.35%	+30.18%	+15.27%
Trades	17	11	28
Win rate	35%	27%	32%
Best trade	OKLO +$21.77	MU +$880.35	—
Worst trade	CCJ -$21.17	SPCX -$103.38	—

7.2 Per-Ticker Results

Ticker	Alpha P&L	Beta P&L	Winner	Key Observation
SPCX	$0.00 (0 trades)	-$103.38	Alpha	Alpha correctly avoided IPO; Beta bought at $160.95, stopped at -10%
OKLO	+$21.77	-$87.58	Alpha	Alpha’s reversion worked; Beta entered too early, stopped out
NVDA	+$4.89	+$50.37	Beta	Both positive; Beta’s thesis caught the move
APLD	+$19.49	+$86.07	Beta	Both positive; thesis + reversion aligned
CCJ	-$21.17	-$71.43	Alpha	Both lost; Alpha’s 3% stop limited damage vs Beta’s 10%
MU	-$16.12	+$880.35	Beta	Beta’s thesis held through volatility for +35.2%

7.3 Behavioral Diversity — The Core PAST Claim

The PAST framework's central claim is that different personality vectors (**P**) produce different behavior on the same instrument. The data confirms this:

SPCX (SpaceX IPO, June 12, 2026): - **Alpha** ($P_4 = 6$, Patience): 0 trades. RSI was 89 on IPO day. The rule cascade from $P_4 = 6$ produced a strict entry filter (RSI < 35 + rising 50-day MA). Alpha correctly sat out. - **Beta** ($P_2 = 7$, Conviction): Bought at $160.95 on IPO day. The rule cascade from $P_2 = 7$ produced a high-conviction, act-on-thesis behavior. Stopped at $144.86 (-10%). - **Result:** Alpha's $P_4 = 6$ kept it out of the IPO euphoria. Beta's $P_2 = 7$ put it in immediately. Same stock, opposite behavior — exactly as the PAST vectors predicted.

MU (Micron, high volatility): - **Alpha** ($P_3 = 2$, Impulsivity): 4 trades, all small. Tight 3% stops ($P_1 = 3$) clipped it on volatility. Net: -16.12. – $**Beta**$(P_1=6$, Risk; $P_2 = 7$, Conviction): 1 trade. Bought near 52-week low, held through volatility for +35.2%. The wide 10% stop ($P_1 = 6$) and high conviction ($P_2 = 7$) captured the full move. Net: +$880.35. - **Result:** Alpha's tight personality vector lost small. Beta's wide personality vector won big. The PAST scores didn't just produce different behavior — they produced predictable, measurable differences in risk-taking and hold duration.

7.4 PAST Score Evolution

Rocky's bi-weekly tuning adjusted scores based on performance, demonstrating the coach review loop ($\Delta P_{i,t}$):

Agent	Score	$P_{i,t}$	$P_{i,t+1}$	ΔP	Reason
Alpha	Patience (P_4)	6	7	+1	Low win rate → tighten entry criteria
Beta	Impulsivity (P_3)	5	6	+1	Stop loss hit on SPCX → increase scrutiny

Alpha's P_4 increase from 6→7 was Rocky's response to CCJ and MU losses. The effect: α_4 increased from 0.83 to 1.0, tightening the entry filter from RSI < 35 to RSI < 20 + bullish confirmation. This is the PAST loop in action — the coach observed losses ($K_{\text{observed}} < K_{\text{target}}$) and shifted the personality vector ($\Delta P_{4,t} = +1$) to reduce risk, all without touching model weights.

7.5 Judge Performance

The Judge's 13-point checklist approved 100% of trades that met entry criteria. No trade was rejected. This indicates the entry rules are well-calibrated — agents only propose trades that pass the checklist. The Judge's IPO ban (Item 11c) would have blocked Beta's SPCX trade in production — the backtest was run before this rule was added.

8. Discussion

8.1 Strengths

Behavioral diversity is emergent and measurable. The same stock produces different behavior from different agents — not because the prompts differ, but because the personality vectors cascade into different trading rules. This is the core contribution: personality as a tunable parameter.

The coaching loop creates adaptation without retraining. Rocky's bi-weekly adjustments evolved Alpha's behavior from a 0-trade regime (too strict) to a 17-trade regime (secondary entry path), then tightened P_4 when losses accumulated. This adaptation happened by changing a single integer ($P_4 : 6 \rightarrow 7$), not by rewriting prompts or fine-tuning models. No GPU. No backpropagation. No checkpoint rollback.

File-based memory provides full auditability. Every trade, every Judge verdict, every PAST adjustment is logged in markdown. The trust region ($|\Delta P| \leq 1$) ensures changes are gradual and reviewable. This is critical for a system handling real capital.

ICLE produces self-correction. Alpha's state file documents 18 strategy learnings accumulated over 6.5 months, including the observation that "the RSI<35 threshold may be too conservative for names with strong intact uptrends." This is the agent learning from its own execution history — $\Psi(H_t, K)$ in action.

8.2 Limitations

Small sample size. 28 trades across 6 tickers is insufficient for statistical significance. The +15.3% return could be partially attributed to the AI capex bull market of H1 2026.

Self-evaluation in simulation mode. Without Claude Code OAuth, the Judge ran as self-evaluation (same LLM evaluating its own theses). Production uses a separate model (Fable 5).

Regime dependence. Beta's +$754 was dominated by a single +$880 MU trade. In a different market regime, Beta's 27% win rate with 10% stops could produce consecutive losses without a compensating win.

8.3 Comparison with Existing Frameworks

Dimension	PAST (Ours)	TradingAgents	AI Hedge Fund	AI-Trader
Agents	4 (competitive)	7 (collaborative)	18 (collaborative)	Configurable
Key innovation	Weight-free personality tuning	Structured debate	Investor personas	Agent-native exec
Adaptation	Bi-weekly PAST tuning (ΔP)	None	None	Unknown

Dimension	PAST (Ours)	TradingAgents	AI Hedge Fund	AI-Trader
ICLE	Yes (file-based log exploitation)	No	No	Unknown
Model diversity	3 models (GLM, Deepseek, Claude)	1 model	13 providers	Any
Independent Judge	Yes (separate LLM)	Risk Manager (same LLM)	Portfolio Manager	Unknown
Local fallback	Deepseek R1 70B (Ollama)	None	None	None
Real trading	Yes (Robinhood, $5K)	Backtest only	Simulation only	Yes (Robinhood)
Backtest return	+15.3% (6.5 months)	+24.9% annualized	Not published	Not published

PAST's unique contribution: No other framework tunes agent personality based on observed performance. TradingAgents and AI Hedge Fund use static roles/personas. PAST makes personality a dynamic, coach-tuned parameter with a trust region for stability, supplemented by ICLE for micro-level self-correction.

8.4 Future Work

1. **Extended backtesting** — 12+ months across 50+ tickers and multiple market regimes
2. **Live deployment results** — compare simulated vs. live P&L after August 2026 go-live
3. **Additional personalities** — test Druckenmiller (macro-adaptive), Tepper (distressed contrarian), Tudor Jones (trend-following) as alternative personality vectors
4. **PAST drift analysis** — measure how personality vectors evolve over 6+ months and correlate with performance
5. **Regime detection** — add a regime classifier that Rocky uses to adjust P proactively
6. **Generalization** — Apply PAST to non-trading domains. The PAST framework is domain-agnostic: any multi-agent system where behavioral diversity and adaptation are valuable can benefit. Potential applications include:
   - **AI tutors for children** — personality tuning for teaching style adaptation (visual vs. reading, patient vs. direct, structured vs. exploratory). A coach agent observes learning outcomes and adjusts the tutor's Likert scores (Patience, Assertiveness, Empathy) bi-weekly, exactly as Rocky adjusts trading agent scores.
   - **Software engineering agents** — tuning code review strictness, refactoring aggressiveness, or documentation thoroughness based on project KPIs (bug rate, deployment speed, code quality).
   - **Customer support** — adjusting agent empathy, technical depth, and escalation thresholds based on satisfaction scores and resolution rates.
   - **Content moderation** — tuning strictness, context sensitivity, and appeal handling based on false positive/negative rates.

---

9. Conclusion

We presented **Personality-Adaptive Score Tuning (PAST)**, a novel, weight-free alignment framework for multi-agent LLM systems. By quantifying agent personas into discrete 1–7 Likert vectors, PAST cleanly decouples runtime behavioral steerability from parameter-level model training. The combination of a macro-level, bi-weekly **Coach review loop** (with trust region) and a micro-level **In-Context Log Exploitation (ICLE)** pipeline yields system performance that rivals parameter-tuned frameworks while introducing zero compute overhead at the training layer.

In a trading system backtest across 6 tickers and 6.5 months, the framework produced +15.3% returns with measurable behavioral diversity — agents with different PAST vectors made different decisions on the same instrument, exactly as their profiles predicted. The coaching loop adapted personality scores bi-weekly based on performance, creating a closed-loop learning system without weight fine-tuning.

The key insight: **agents learn from their PAST.** Not by updating weights, but by adjusting the quantified personality that drives their rules, and by exploiting their own execution history at the inference boundary. This is contextual reinforcement learning at the instruction level — lightweight, interpretable, auditable, and achievable on consumer hardware without GPU training.

PAST offers engineers total system explainability, immediate behavioral reversibility, and absolute immunity against catastrophic forgetting. This work confirms that structuring and tuning personality metrics at the inference boundary is a highly viable, energy-efficient alternative to gradient-descent alignment strategies for agentic AI.

References

1. Xiao, Y., Sun, E., Luo, D., Wang, W. (2024). “TradingAgents: Multi-Agents LLM Financial Trading Framework.” arXiv:2412.20138.
2. Singh, V. (2025). “AI Hedge Fund.” GitHub: virattt/ai-hedge-fund.
3. Fan, T. et al. (2026). “AI-Trader: 100% Fully-Automated Agent-Native Trading.” GitHub: HKUDS/AI-Trader.
4. Schulman, J. et al. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347. (Trust region inspiration)
5. Rafailov, R. et al. (2023). “Direct Preference Optimization.” arXiv:2305.18290.
6. Hartley, M. et al. (2025). “Personality Traits Shape LLM Risk-Taking Behavior.”
7. Wang, Q. et al. (2025). “Risk Profiling and Modulation for Large Language Models.”
8. Zhipu AI (2026). “GLM-5.2 Technical Report.” arXiv:2602.15763.
9. Torres-Leguet, J. (2025). “The Little Book of Reinforcement Learning.”
10. Grable, J. & Lytton, R. (2003). “Financial Risk Tolerance Revisited.” (Likert scale inspiration)
11. Microsoft (2024). “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.”
12. CrewAI (2024). “CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents.”
13. LangGraph (2024). “LangGraph: Building Stateful Multi-Actor Applications with LLMs.”

This research is for educational purposes. Past performance does not indicate future results. The Trading Arena uses real capital and accepts the risks of AI-driven trading.

Code and backtest harness available at [GitHub repository — to be published August 6, 2026].